Scenarios

In this section, we cover the six key scenarios that are common in many serverless applications and how they influence the design and architecture of your serverless application workloads on AWS. We will present the assumptions we made for each of these scenarios, the common drivers for the design, and a reference architecture of how these scenarios should be implemented.

Key scenarios

- RESTful microservices
- Alexa skills
- Mobile backend
- Streaming processing
- Web application
- Event-driven architectures

RESTful microservices

When building a microservice, think about how a business context can be delivered as a reusable service for your consumers. The specific implementation will be tailored to individual use cases, but there are several common themes across microservices to ensure that your implementation is secure, resilient, and constructed to give the best experience for your customers.

Building serverless microservices on AWS enables you to not only take advantage of the serverless capabilities themselves, but also to use other AWS services and features, as well as the ecosystem of AWS and AWS Partner Network (APN) tools. Serverless technologies are built on top of fault-tolerant infrastructure, enabling you to build reliable services for your mission-critical workloads. The ecosystem of tooling enables you to streamline the build, automate tasks, orchestrate dependencies, and monitor and govern your microservices. Lastly, AWS serverless tools are pay-as-you-go, enabling you to grow the service with your business and keep your costs down during entry phases and non-peak times.

Characteristics

- You want a secure, easy-to-operate framework that is simple to replicate and has high levels of resiliency and availability.

- You want to log utilization and access patterns to continually improve your backend to support customer usage.
- You are seeking to use managed services as much as possible for your platforms, which reduces the heavy lifting associated with managing common platforms, including security and scalability.

Reference architecture

Figure 1: Reference architecture for RESTful microservices

1. **Customers** leverage your microservices by making HTTP API calls. Ideally, your consumers should have a tightly bound service contract to your API to achieve consistent expectations of service levels and change control.
2. **Amazon API Gateway** hosts RESTful HTTP requests and responses to customers. In this scenario, API Gateway provides built-in authorization, throttling, security, fault tolerance, request and response mapping, and performance optimizations.
3. **AWS Lambda** contains the business logic to process incoming API calls and use DynamoDB as a persistent storage.
4. **Amazon DynamoDB** persistently stores microservices data and scales based on demand. Since microservices are often designed to do one thing well, a schemaless NoSQL data store is regularly incorporated.

Configuration notes

- Use API Gateway logging to understand visibility of microservices consumer access behaviors. This information is visible in Amazon CloudWatch Logs and can be quickly viewed through Log Pivots, analyzed in CloudWatch Logs Insights or fed into other searchable engines such as OpenSearch Service or Amazon S3 (with Amazon Athena). The information delivered gives key visibility, such as:

- Understanding common customer locations, which may change geographically based on the proximity of your backend.
- Understanding how customer input requests may have an impact on how you partition your database.
- Understanding the semantics of abnormal behavior, which can be a security flag.
- Understanding errors, latency, and cache hits or misses to optimize configuration.
- This model provides a framework that is easy to deploy and maintain, and a secure environment that will scale as your needs grow.

Alexa skills

The Alexa Skills Kit gives developers the ability to extend Alexa's capabilities by building natural and engaging voice and visual experiences. Successful skills are habit-forming, where users routinely come back because it offers something unique, it provides value in new, novel, and frictionless ways.

The biggest cause of frustration from users is when the skill doesn't behave as expected. It's essential to start by designing a voice interaction model and working backwards since some users may say too little, too much, or possibly something unexpected. The voice design process involves creating, scripting, and planning for expected as well as unexpected utterances.

Example Scripts

User: Alexa, open Paragon Investments.

Alexa: Welcome to Paragon. You can get a portfolio update or a security quote. Which one would you like?

User: Portfolio update.

Alexa: Your portfolio grew by 0.27% today. A total of $322. Would you like a breakdown of the changes?

User: Yes.

Alexa: Crystal Cleaner grew by 0.35% for a total gain of $165.03 (half second pause) Pizza Barn is up by 0.21% for a total gain of $157. (half second pause) And Belle's Boats is up 1.41% for a gain of $27.68. (half second pause) Do you want any other portfolio details?

User: That's it.

Alexa: What else would you like help with today!

User: Nothing, thanks.

Alexa: Okay, have a nice day.

User: Alexa, ask Paragon how my investments did today.

Alexa: Your portfolio grew by 0.27% today. A total of $322. Would you like a breakdown of you investments?

User: Yeah.

Alexa: Crystal Cleaner grew by 0.35% for a total gain of $165.03 (half second pause) Pizza Barn is up by 0.21% for a total gain of $157. (half second pause) And Belle's Boats is up 1.41% for a gain of $27.68. (half second pause) Do you want any other portfolio details?

User: Exit.

Alexa: Okay, have a nice day.

Figure 2: Alexa Skill example design script

With a basic script in mind, you can use the following techniques before start building a skill:

- **Outline the shortest route to completion.**
  - The shortest route to completion is generally when the user gives all information and slots at once, an account is already linked if relevant, and other prerequisites are satisfied in a single invocation of the skill.
- **Outline alternate paths and decision trees.**
  - Often, what the user says doesn't include all information necessary to complete the request. In the flow, identify alternate pathways and user decisions.
- **Outline behind-the-scenes decisions the system logic will have to make.**
  - Identify behind-the-scenes system decisions, for example with new or returning users. A background system check might change the flow a user follows.
- **Outline how the skill will help the user.**
  - Include clear directions in the help for what users can do with the skill. Based on the complexity of the skill, the help might provide one simple response or many responses.
- **Outline the account linking process, if present.**
  - Determine the information that is required for account linking. You also need to identify how the skill will respond when account linking hasn't been completed.

Characteristics

- You want to create a complete serverless architecture without managing any instances or servers.
- You want your content to be decoupled from your skill as much as possible.
- You are looking to provide engaging voice experiences exposed as an API to optimize development across wide-ranging Alexa devices, Regions, and languages.
- You want elasticity that scales up and down to meet the demands of users and handles unexpected usage patterns.

Reference architecture

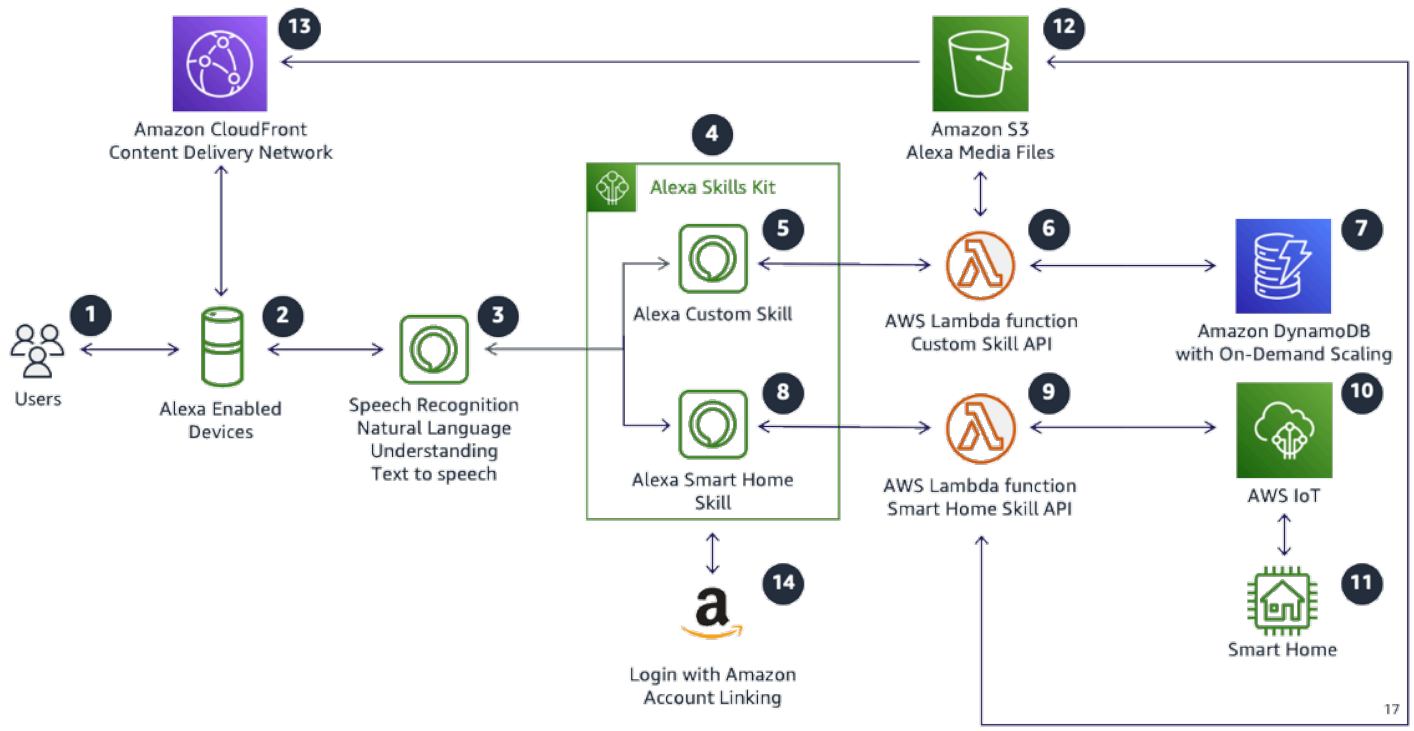

Figure 3: Reference architecture for an Alexa Skill

1. **Alexa users** interact with Alexa skills by speaking to Alexa-enabled devices using voice as the primary method of interaction.
2. **Alexa-enabled devices** listen for a wake word and activate as soon as one is recognized. Supported wake words are Alexa, Computer, and Echo.
3. The **Alexa Service** performs common Speech Language Understanding (SLU) processing on behalf of your Alexa Skill, including Automated Speech Recognition (ASR), Natural Language Understanding (NLU), and Text to Speech (TTS) conversion.
4. **Alexa Skills Kit (ASK)** is a collection of self-service APIs, tools, documentation, and code examples that make it fast and easy for you to add skills to Alexa. ASK is a trusted AWS Lambda trigger, allowing for seamless integration.
5. **Alexa Custom Skill** gives you control over the user experience, allowing you to build a custom interaction model. It is the most flexible type of skill, but also the most complex.
6. A **Lambda function** using the Alexa Skills Kit, allowing you to seamlessly build skills avoiding unneeded complexity. Using it you can process different types of requests sent from the Alexa Service and build speech responses.

7. A **DynamoDB Database** can provide a NoSQL data store. Using DynamoDB's on-demand scaling mechanism offers simple pay-per-request pricing for read and write requests so that you only pay for what you use, and you do not need to worry about forecasting read and write throughput. DynamoDB is commonly used by Alexa skills to persist user state and sessions.
8. **Alexa Smart Home Skill** allows you to control devices such as lights, thermostats, and smart TVs using the Smart Home API. Smart Home skills are simpler to build than custom skills since they don't give you control over the interaction model.
9. A **Lambda function** is used to respond to device discovery and control requests from the Alexa Service. Developers use it to control a wide-ranging number of devices including entertainment devices, cameras, lighting, thermostats, locks, and many more.
10. **AWS IoT** allows developers to securely connect their devices to AWS and control interaction between their Alexa skill and their devices.
11. An Alexa-enabled **Smart Home** can have an unlimited number of IoT connected devices receiving and responding and to directives from an Alexa Skill.
12. **Amazon S3** stores your skills static assets including images, content, and media. Its contents are securely served using CloudFront.
13. **Amazon CloudFront Content Delivery Network (CDN)** provides a CDN that serves content faster to geographically distributed mobile users and includes security mechanisms to static assets in Amazon S3.
14. **Account Linking** is needed when your skill must authenticate with another system. This action associates the Alexa user with a specific user in the other system.

Configuration notes

- Validate Smart Home request and response payloads by validating against the JSON schema for all possible Alexa Smart Home messages sent by a skill to Alexa.
- Ensure that your Lambda function timeout is less than eight seconds and can handle requests within that timeframe. (The Alexa Service timeout is eight seconds.)
- Follow best practices when creating your DynamoDB tables. Use on-demand tables when you are not certain how much read or write capacity you need. You can use provisioned capacity with automatic scaling enabled if you know read and write capacity and do not expect large spikes in traffic. For Skills that are heavy on reads, DynamoDB Accelerator(DAX) can greatly improve response times.

- Account linking can provide user information that may be stored in an external system. Use that information to provide contextual and personalized experience for your user. Alexa has guidelines on Account Linking to provide frictionless experiences.
- Use the skill beta testing tool to collect early feedback on skill development, and for skills versioning, to reduce impact on skills that are already live.
- Use ASK CLI to automate skill development and deployment.

Mobile backend

Users increasingly expect their mobile applications to have a fast, consistent, and feature-rich user experience. At the same time, mobile user patterns are dynamic with unpredictable peak usage and often have a global footprint.

The growing demand from mobile users means that applications need a rich set of mobile services that work together seamlessly without sacrificing control and flexibility of the backend infrastructure. Certain capabilities across mobile applications, are expected by default:

- Ability to query, mutate, and subscribe to database changes.
- Offline persistence of data and bandwidth optimizations when connected.
- Search, filtering, and discovery of data in applications.
- Analytics of user behavior.
- Targeted messaging through multiple channels (Push Notifications, SMS, Email).
- Rich content such as images and videos.
- Data synchronization across multiple devices and multiple users.
- Fine-grained authorization controls for viewing and manipulating data.

Building a serverless mobile backend on AWS enables you to provide these capabilities while automatically managing scalability, elasticity, and availability in an efficient and cost effective way.

Characteristics

- You want to control application data behavior from the client and explicitly select what data you want from the API.
- You want your business logic to be decoupled from your mobile application as much as possible.

- You are looking to provide business functionalities as an API to optimize development across multiple platforms.
- You are seeking to use managed services to reduce undifferentiated heavy lifting of maintaining mobile backend infrastructure while providing high levels of scalability and availability.
- You want to optimize your mobile backend costs based upon actual user demand instead of paying for idle resources.

Reference architecture

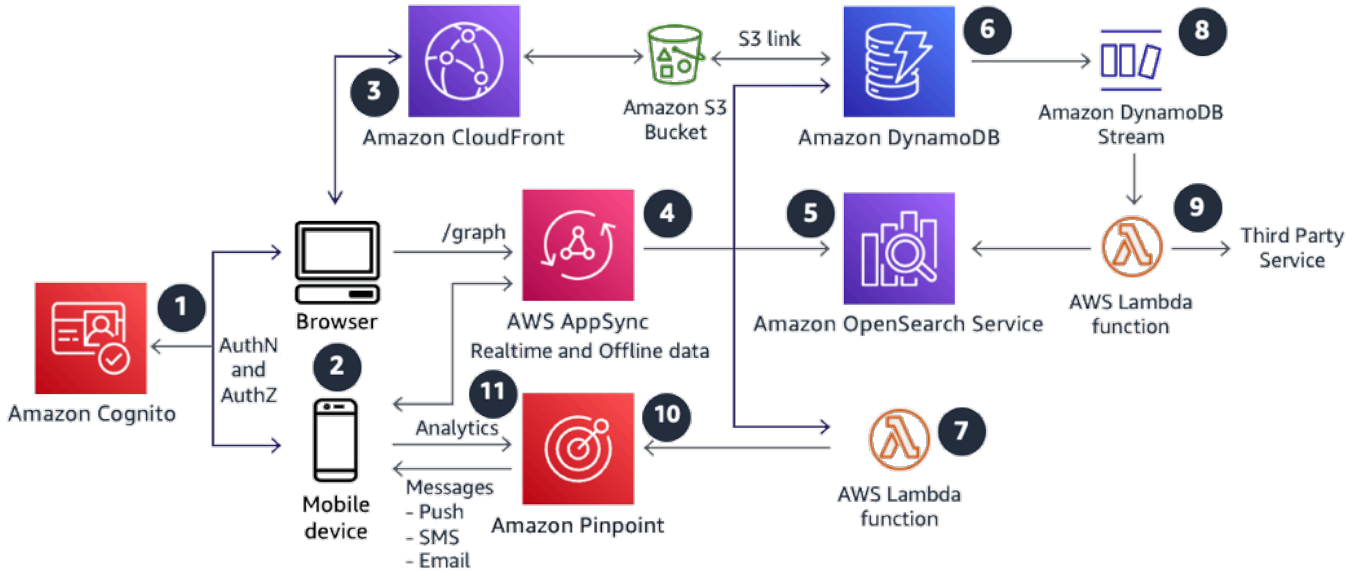

Figure 4: Reference architecture for a mobile backend

1. **Amazon Cognito** is used for user management and as an identity provider for your mobile application. Additionally, it allows mobile users to leverage existing social identities such as Facebook, Twitter, Google+, and Amazon to sign in.
2. **Mobile users** interact with the mobile application backend by performing GraphQL operations against AWS AppSync and AWS service APIs (for example, Amazon S3 and Amazon Cognito).
3. **Amazon S3** stores mobile application static assets including certain mobile user data such as profile images. Its contents are securely served via CloudFront.
4. **AWS AppSync** hosts GraphQL HTTP requests and responses to mobile users. In this scenario, data from AWS AppSync is in real-time when devices are connected, and data is available offline

as well. Data sources for this scenario are Amazon DynamoDB, Amazon OpenSearch Service, or AWS Lambda functions.

5. **Amazon OpenSearch Service** acts as a main search engine for your mobile application as well as analytics.
6. **Amazon DynamoDB** provides persistent storage for your mobile application, including mechanisms to expire unwanted data from inactive mobile users through a **Time to Live (TTL)** feature.
7. An **AWS Lambda** function handles interaction with other third-party services, or calling other AWS services for custom flows, which can be part of the GraphQL response to clients.
8. **Amazon DynamoDB Streams** captures item-level changes and enables a Lambda function to update additional data sources.
9. An **AWS Lambda** function manages streaming data between DynamoDB and OpenSearch Service, allowing customers to combine data sources logical GraphQL types and operations.
10. **Amazon Pinpoint** captures analytics from clients, including user sessions and custom metrics for application insights.
11. **Amazon Pinpoint** delivers messages to all users or devices, or a targeted subset based on analytics that have been gathered. Messages can be customized and sent using push notifications, email, or SMS channels.

Configuration notes

- Performance test your Lambda functions with different memory and timeout settings to ensure that you're using the most appropriate resources for the job.
- Follow best practices when creating your DynamoDB tables and consider having AWS AppSync automatically provision them from a GraphQL schema, which will use a well-distributed hash key and create indexes for your operations. Make certain to calculate your read and write capacity, and table partitioning to ensure reasonable response times.
- Use the AWS AppSync server-side data caching to optimize your application experience, as all subsequent query requests to your API will be returned from the cache, which means data sources won't be contacted directly unless the TTL expires.
- Follow best practices when managing Amazon OpenSearch Service domains. Additionally, Amazon OpenSearch Service provides an extensive guide on designing concerning sharding and access patterns that also apply here.

- Use the fine-grained access controls of AWS AppSync, configured in resolvers, to filter GraphQL requests down to the per-user or group level if necessary. This can be applied to AWS Identity and Access Management (IAM) or Amazon Cognito user pools authorization with AWS AppSync.
- Use AWS Amplify and Amplify CLI to compose and integrate your application with multiple AWS services. Amplify Console also takes care of deploying and managing stacks.

For low-latency requirements where near-to-none business logic is required, Amazon Cognito Federated Identity can provide scoped credentials so that your mobile application can talk directly to an AWS service, for example, when uploading a user's profile picture, retrieve metadata files from Amazon S3 scoped to a user.

Streaming processing

Ingesting and processing real-time streaming data requires scalability and low latency to support a variety of applications such as activity tracking, transaction order processing, click-stream analysis, data cleansing, metrics generation, log filtering, indexing, social media analysis, and IoT device data telemetry and metering. These applications are often spiky and process thousands of events per second.

Using AWS Lambda and Amazon Kinesis, you can build a serverless stream process that automatically scales without provisioning or managing servers. Data processed by AWS Lambda can be stored in DynamoDB and analyzed later.

Characteristics

- You want to create a complete serverless architecture without managing any instance or server for processing streaming data.
- You want to use the Amazon Kinesis Producer Library (KPL) to take care of data ingestion from a data producer-perspective.

Reference architecture

Here we are presenting a scenario for common stream processing, which is a reference architecture for analyzing social media data:

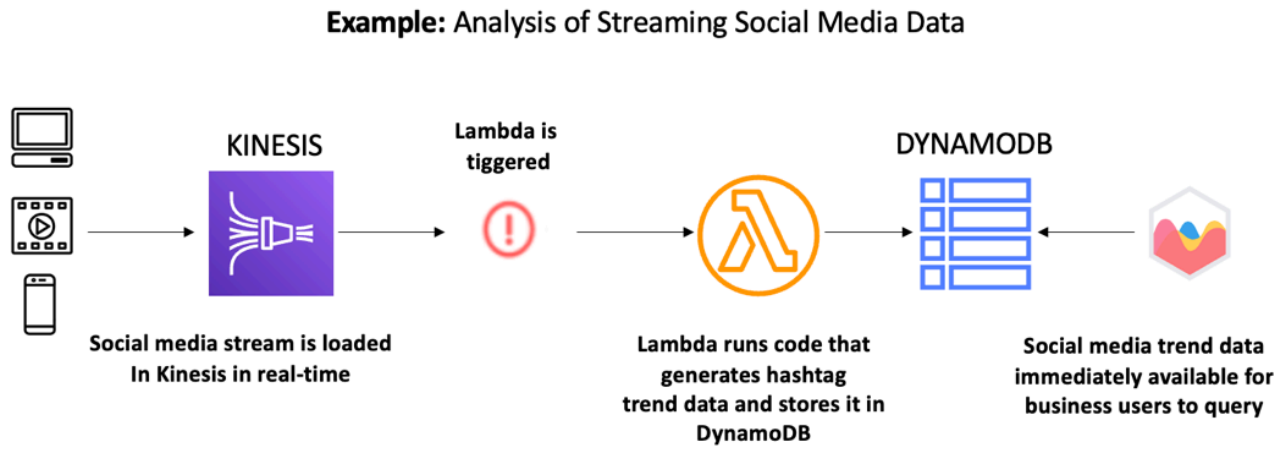

Figure 5: Reference architecture for stream processing

1. **Data producers** use the Amazon Kinesis Producer Library (KPL) to send social media streaming data to a Kinesis stream. Amazon Kinesis Agent and custom data producers that leverage the Kinesis API can also be used.
2. An **Amazon Kinesis stream** collects, processes, and analyzes real-time streaming data produced by data producers. Data ingested into the stream can be processed by a consumer, which, in this case, is Lambda.
3. **AWS Lambda** acts as a consumer of the stream that receives an array of the ingested data as a single event or invocation. Further processing is carried out by the Lambda function. The transformed data is then stored in a persistent storage, which, in this case, is DynamoDB.
4. **Amazon DynamoDB** provides a fast and flexible NoSQL database service including triggers that can integrate with AWS Lambda to make such data available elsewhere.
5. **Business users** can use a reporting interface on top of DynamoDB to gather insights out of social media trend data.

Configuration notes

- Consider reviewing the Streaming Data Solutions whitepaper for batch processing, analytics on streams, and other useful patterns.
- Consider using Firehose over Lambda when ingested data needs to be continuously loaded into Amazon S3, Amazon Redshift, or Amazon OpenSearch Service.

- Consider using Managed Service for Apache Flink over Lambda when standard SQL or Apache Flink could be used to query streaming data, and load only its results into Amazon S3, Amazon Redshift, OpenSearch Service, or Kinesis Data Streams.
- Use Lambda maximum retry attempts, maximum record age, bisect batch on function error, and on-failure destination error controls to build more resilient stream processing applications. In addition consider using Lambda's custom checkpoint feature, where you can have more precise control over how you choose to process batches containing failed messages.
- Duplicated records may occur, and you must use both retries and idempotency within your application for both consumers and producers.
- Follow best practices for AWS Lambda stream-based invocation since that covers the effects on batch size, concurrency per shard, and monitoring stream processing in more detail.
- When not using KPL, make certain to take into account partial failures for non-atomic operations, such as `PutRecords`, since the Kinesis API returns both successfully and unsuccessfully processed records upon ingestion time.
- If you are using Kinesis Data Streams in **provision capacity mode**, follow best practices when re-sharding Kinesis Data Streams to accommodate a higher ingestion rate. Concurrency for stream processing is dictated by the number of shards and by the parallelization factor. Therefore, adjust it according to your throughput requirements.
- Consider using filtering event sources for AWS Lambda functions. Event filtering helps reduce requests made to your Lambda functions, may simplify code, and can reduce overall cost. At the time of writing, event filtering is only natively supported in CloudFormation and not in AWS CDK. If you are using AWS CDK you can still support Lambda Event filtering by using Escape Hatches to extend some functionality not directly available in CDK constructs.
- Consider using tumbling windows for AWS Lambda functions when you need to continuously calculate aggregates over a time period, such as 30-second averages. This feature allows you to do these types of aggregates without implementing a temporary datastore or using complicated streaming analytics frameworks.

Web application

Web applications often have demanding requirements to ensure a consistent, secure, and reliable user experience. Workloads which need to scale to thousands or millions of users require provisioning infrastructure for peak loads or sophisticated auto-scaling mechanisms, when available. On-premises workloads require significant capital expenditures and long lead times for capacity provisioning.

By taking a serverless-first approach on AWS you free yourself from the burden of managing servers, perfecting auto-scaling policies or paying for idle resources. Serverless workloads on AWS can provide the same, or better, security, reliability or performance when compared with server-based workloads.

Characteristics

- You want a scalable, resilient, and highly-available web application that can go global in minutes.
- You are seeking to reduce operational overhead by using managed services.
- You want to optimize your costs based on user demand and usage, instead of paying for idle resources.
- You want to create a framework that is easy to set up and operate, and that you can extend with limited impact later.

Reference architecture

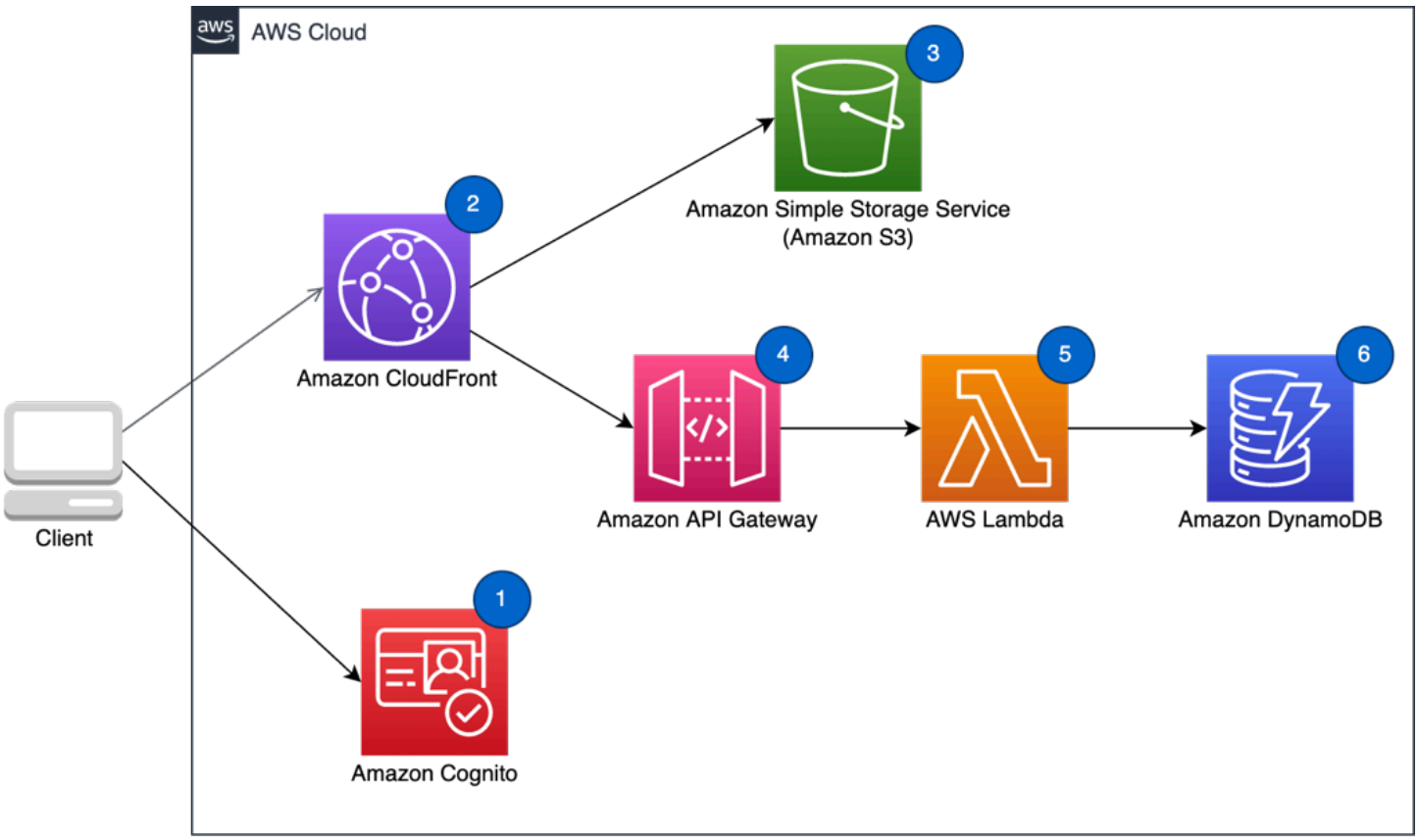

Figure 6: Reference architecture for a web application

1. **Amazon Cognito user pools** provides user management and identity provider features for your web application. Tokens issued by Amazon Cognito are used to authenticate users when making request to Amazon API Gateway.
2. **Amazon CloudFront** provides a better user experience by accelerating content delivery of static assets and calls to your backend compute layer. CloudFront brings content closer to clients using AWS's global Points of Presence (PoPs). CloudFront can also cache API calls to reduce calls to compute backends while also providing optimal network routing for non-cacheable API calls.
3. **Amazon S3** hosts web application static assets such as HTML, CSS, JavaScript and images. Content is securely served through CloudFront.
4. **Amazon API Gateway** serves as the secure HTTPS endpoint. Web applications make REST API calls to a public HTTPS endpoint using either a custom domain name or a unique API Gateway-provided domain.
5. An **AWS Lambda** function provides create, read, update, and delete (CRUD) operations on top of DynamoDB for your web application.
6. **Amazon DynamoDB** can provide a NoSQL data store which elastically scales with your web application.

Configuration notes

- Follow best practices for deploying your serverless web application frontend on AWS. More information can be found in the operational excellence pillar.
- For single-page web applications you can use AWS Amplify Hosting to manage atomic deployments, cache expiration and custom domains.
- Refer to the security pillar for recommendations on authentication and authorization.
- Refer to the RESTful Microservices scenario for recommendations on web application backend.
- For web applications that offer personalized services, you can use API Gateway usage plans. You can use Amazon Cognito user pools to scope users to specific resources or functionality. For example, a premium user may have higher throughput for API calls, access to additional APIs and additional storage.
- Refer to the Mobile Backend scenario if your application uses search capabilities that are not covered in this scenario.

Event-driven architectures

Event-driven architectures are becoming a popular and preferable way of building large distributed microservice-based applications. This approach helps you build scalable, resilient, agile and cost-effective solutions.

Using AWS serverless services to implement event-driven approach will allow you to build scalable, fault tolerant applications. You can use messaging services like Amazon SQS for reliable and durable communication between microservices. For fan out of the events you can use Amazon SNS topics. If you need event filtering and routing you can utilize Amazon EventBridge.

Typical use cases for event-driven architectures:

- Communication between microservices
- Integration with third-party SaaS applications
- Cross-account and cross region data replication
- Parallel event processing, fanout

Reference architecture

Every event-driven architecture consists of three main parts:

- Event sources
- Event routers
- Event destinations

The most common event sources could be other AWS services, your microservices or applications, or third-party SaaS offerings. For routing those events, you can create rules matching specific parts of the event and provide destinations of where to send them. You describe the rules and destinations with the help of Amazon EventBridge.

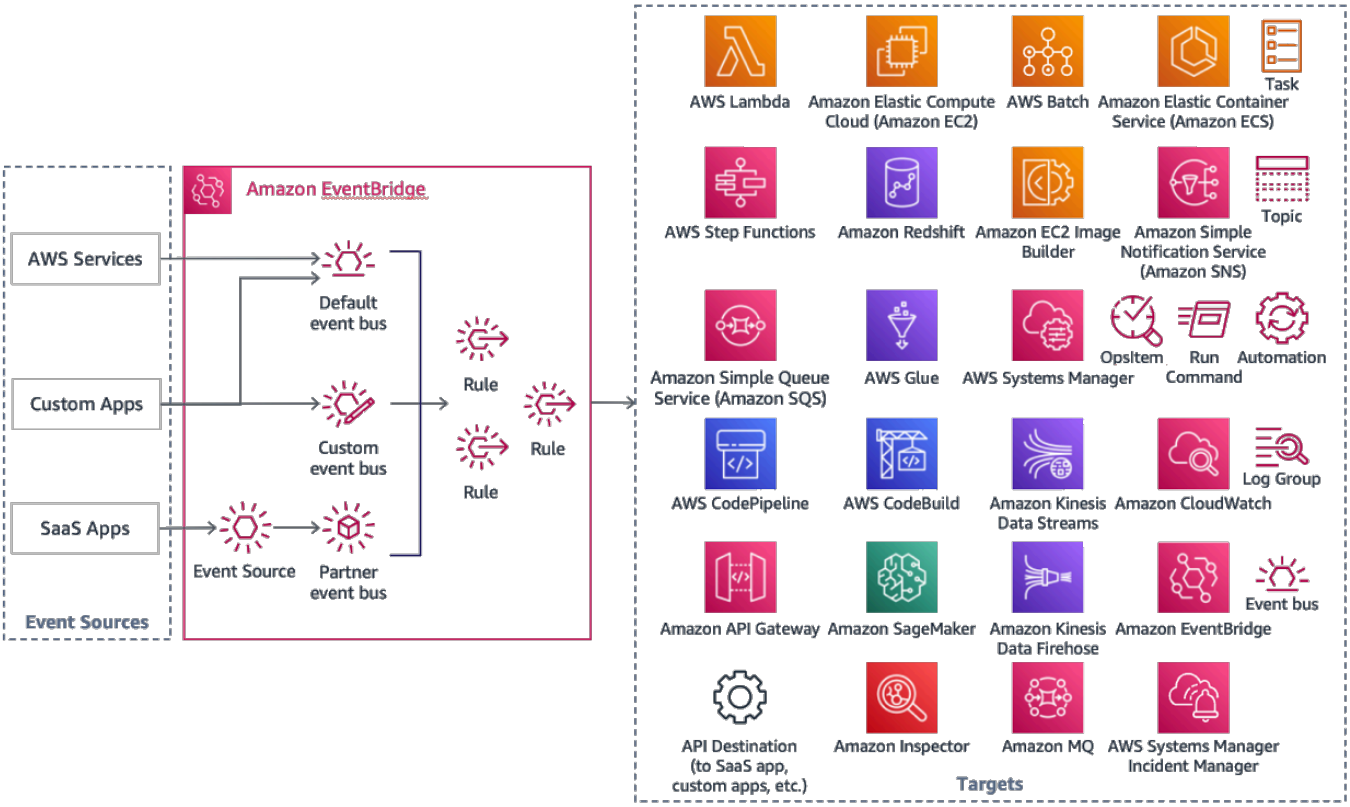

Figure 7: Reference architecture for EventBridge deployment

When you are building event-driven microservices applications, it's important to agree on the data contract for the event producers and consumers. This will help to validate the events and automatically generate bindings for the used programming language. Amazon EventBridge allows you to use schemas in OpenAPI 3 and JSONSchema Draft4 formats.

As all the event-driven applications are distributed it is important to use tracing to understand and observe service dependencies and diagnose any bottlenecks and issues in the application. To use tracing, you can enable integration between EventBridge and AWS X-Ray.

Configuration notes

EventBridge can introduce additional latency to the application. If latency is a concern, consider using Amazon SNS and Amazon SQS for event filtering and routing.